Aran: normaliser, registries and typo repair

Roadmap items 5 and 2, built and measured. This closes the two Tier 1 items the roadmap said to do first, because they make every later number better.

What	File	Owners
Normaliser: undoes encodings, other-script digits, separators, split fields and base64 before detection, and maps every finding back to the original line	`research/firewall/normalise.py`	Inbathamizhan, Aasif
Own-customer registry (HMAC in a cuckoo filter), trace-bound OTP registry, typo repair	`research/firewall/registry.py`	Inbathamizhan, Aasif
Registry evidence in the calibrated detector (Bayes' rule on top of the calibrated probability)	`research/ai/detector.py`	Bharadhwaj, Arun
In-place replacement for split fields and base64 blobs; own-token check	`research/firewall/aran_firewall.py`	Inbathamizhan, Aasif
Tests (no downloads needed)	`research/test_registry.py`	all
Experiments E10 to E12 and raw results	`research/experiment_registry.py`, `results_registry.json`	all

Read first: Learn 3 section 2.6 (HMAC), Learn 4 sections 4.2 and 4.3 (Bloom and cuckoo filters) and the Bayes section, Learn 2 (why Verhoeff catches every single typo).

1. Headline results

Same data and splits as the AI and immune layers report: 16,000 real log lines from 8 Loghub systems the models never saw, with 1,895 generated sensitive values injected into 1,110 of them. Nothing here uses a real person's identifier.

Test	Before (detector only)	After (normaliser and registries)
Sensitive values removed by the firewall, all types	83.5%	**89.7%**
Mobiles removed (the weak spot with no cue word)	69.8%	**100%** (94.2% if only 70% of leaked numbers are the bank's own customers)
OTPs removed (the weak spot next to transaction words)	79.4%	**100%**
Identifiers hidden 12 ways (other scripts, escapes, separators, base64, split fields)	8.3%	**100%** (80.3% with the normaliser alone)
Aadhaar and card numbers with one typo	0% removed	**99.2%**
Clean real lines wrongly changed	122 of 14,890	122 of 14,890 (no new false alarms)
Aran tokens wrongly flagged	0	0
Throughput (one laptop CPU core, Python)	298 lines per second	292 lines per second

The two weak spots named in the AI report are closed for any bank that registers its own customers and puts a trace ID in its logs. Names (73.6%) and addresses (89.8%) are unchanged; they are roadmap items 7 and 8.

2. How it works

2.1 The normaliser

Every line becomes one or more views that the detector scans. Each character of a view remembers which characters of the original line it came from, so a finding can be replaced in place.

Hidden as	Example	What the normaliser does
Other-script digits	`४९१४ १७७७ ६३११`, `௪௯௧௪...`, `４９１４...`	Any Unicode decimal digit becomes 0 to 9
Escapes	`%34%39%31...`, `49...`, `&#52;&#57;...`	Decoded, mapped back to the escaped bytes
Separators	`4914.1777.6311`, `4914_1777_6311`, `4 9 1 4 1 7 7 7 6 3 1 1`, `98765 43210`	Dropped, but only for digit-group shapes identifiers really have (4-4-4, 4-4-4-4, 4-6-5, 5-5, 3-3-4, one digit at a time)
Zero-width characters	`491417776311`	Dropped
Split fields	`aadhaar_1=4914 aadhaar_2=1777 aadhaar_3=6311`	Three or four nearby 4-digit groups are joined into one candidate; it counts only if the checksum passes **and** the line names the type or the registry knows the value, so ordinary numbers are left alone. Each part is replaced by 4 digits of the token, so the fields keep their shape
Base64	`payload=dWlkPTQ5MTQxNzc3NjMxMQ==`	Decoded if it is readable text with a digit in it, scanned, cleaned and re-encoded with the same alphabet and padding

2.2 The own-customer registry

The bank adds HMAC(K, type | value) for every Aadhaar, card, mobile and PAN it holds to a cuckoo filter (Fan et al. 2014). The filter stores a 32-bit fingerprint in one of two 4-slot buckets, so:

- it can delete, which DPDP erasure needs and a Bloom filter cannot do;
- the false-positive rate is at most 2 × 4 / 2^32 ≈ 1.9 × 10^-9;
- without the key K the filter reveals nothing; the key belongs in an HSM.

A hit multiplies the detector's odds by the likelihood ratio 1/ε ≈ 5 × 10^8, so a bare `9876543210` with no cue word goes from a low probability to almost 1. A miss is neutral by default, because a leaked number can belong to another bank's customer. A bank that is confident its registry covers every number it could leak can set a strict miss rule (likelihood ratio 0.05 for cards and Aadhaar).

2.3 The trace-bound OTP registry

The OTP service registers HMAC(K, otp | trace ID) with a time to live when it issues an OTP. A 4 to 8 digit number in a log line is an OTP leak if its HMAC with that line's trace ID (the W3C `traceparent` of the request) is live. Without the trace binding, a random 6-digit number would match one of 70,000 live OTPs 7% of the time; with it, a number matches only if it equals the OTP issued in that same request. The firewall takes the trace ID as a parameter: `fw.process(line, src, sink, trace_id=...)`.

2.4 Typo repair and the own-token check

A real Aadhaar with one wrong digit fails Verhoeff and would look like an Aran token, so the detector would ignore it (Theorem 1's hard gate). Verhoeff catches every single-digit error and every adjacent swap, so for a failing 12-digit number Aran tries the 108 single-digit edits and 11 swaps, keeps the ones that pass the checksum (about 14), and asks the registry about each. A registered neighbour means a leak, and the value is tokenised as that customer, so the token is the same as for the correct number.

There is one trap: a token is pseudorandom, so it can also sit one edit away from some other registered customer. With about 14 checksum-valid neighbours and N registered numbers out of about 9 × 10^10, the chance is 14 N / 9 × 10^10, which is **1.5% of all tokens at N = 10^8**. Before flagging a typo, Aran therefore checks whether the number is its own token: the wrong check digit must equal one sink's keyed offset (which rules out 8 of 9 sinks for free) and the decrypted body must be a registered customer.

3. Results in detail

E10. Hidden identifiers

700 TEST lines with 783 Aadhaar, card and mobile values, each hidden 12 ways. A value counts as removed if its digits no longer appear after undoing escapes, other scripts, separators and base64 on the output line.

Hidden as	Today (no normaliser)	Normaliser	Normaliser and registry
plain (control)	629/783	629/783	783/783
Devanagari digits	0/783	629/783	783/783
Tamil digits	0/783	629/783	783/783
full-width digits	0/783	629/783	783/783
URL percent-encoding	0/783	629/783	783/783
JSON \u escapes	0/783	629/783	783/783
HTML entities	0/783	629/783	783/783
dots between groups	0/783	629/783	783/783
underscores between groups	0/783	629/783	783/783
space between every digit	155/783	629/783	783/783
zero-width space inside	0/783	629/783	783/783
split into fields	629/783	629/783	783/783
base64 of the line	0/783	629/783	783/783
Average over the 12 hiding tricks	**8.3%**	**80.3%**	**100.0%**
Clean real lines changed (16,000 lines, unseen systems)	60/16000	60/16000	60/16000
Lines per second	478	517	498

With the normaliser alone, every hidden value is caught exactly as often as the same value written plainly (629 of 783): the 154 left are mobiles with no cue word, which the registry then catches. The normaliser changed no extra real lines. Mobiles are not split into fields (a 10-digit number has no 4-digit groups), so that row counts them in plain form. The first run of E10 caught only 302 values spaced one digit apart, because the spaced-digit rule needed 12 digits and missed 10-digit mobiles; the rule now accepts 10, and the table is from the re-run.

E11. Registries in the full firewall run

Configuration	All values removed	Mobile	OTP	Clean real lines changed	Lines per second
Today (detector only)	83.5%	69.8% (57 of 189 left)	79.4% (60 of 291 left)	122 / 14,890	298
Normaliser and registries, full coverage	89.7%	100%	100%	122 / 14,890	292
Normaliser and registries, 70% coverage	89.1%	94.2% (11 left)	100%	122 / 14,890	290
Same, strict miss rule for cards and Aadhaar	89.7%	100%	100%	121 / 14,890	283

The registry held the injected Aadhaar, card, mobile and PAN values plus 200,000 decoy customers of each type (800,399 entries, 4.0 MB, 16.8 µs per lookup in Python, built in 33 s). The OTP registry held every injected OTP under its line's trace ID plus 70,000 other live OTPs under other trace IDs; none of those 70,000 caused a false alarm. Aadhaar, card and PAN were already at 100% and stay there.

Card false alarms on the real logs of unseen systems: 2 with the detector, 2 with a neutral registry miss, 1 with the strict miss rule. The 81 HDFS block IDs that passed Luhn in E1 were already removed by the calibrated detector in E7, so the registry has little left to remove on this data; its value here is for mobiles and OTPs.

E12. Typo repair

1,990 registered identifiers with one wrong digit or one adjacent swap (Luhn misses some swaps such as 09 and 90; those 10 cases were left out because no checksum can see them).

Case	Without registry	With registry
Aadhaar, wrong digit	0 / 482	474 / 482
Aadhaar, adjacent swap	0 / 513	504 / 513
Card, wrong digit	0 / 505	505 / 505
Card, adjacent swap	0 / 490	490 / 490
Total	**0%**	**99.2%**

The 17 Aadhaar misses are typos that turn the first digit into 0 or 1, or swap it, which no real Aadhaar has, so the number no longer looks like an Aadhaar at all.

Tokens: 5,000 tokens for registered customers over 40 sinks, with 802,000 registered identifiers. Flagged with the own-token check: 0. Without it: also 0 at this registry size, as expected (14 × 8 × 10^5 / 9 × 10^10 ≈ 0.01%). Projected without the check at a bank with 10^8 registered identifiers: 1.5% of tokens. Typo false alarms on 16,000 real log lines: 0.

4. Limits, stated plainly

- **Coverage.** The registries help only for numbers the bank holds and OTPs it issued with a trace ID. A bank without trace IDs in its logs falls back to the context classifier for OTPs (79.4%).
- **Speed.** The Python prototype runs at about 290 lines per second. A cuckoo-filter lookup is 17 µs in Python and well under 1 µs in Rust; roadmap item 12 is the port.
- **The registry is sensitive.** It is a list of every customer, protected only by the HMAC key. The key must live in an HSM, with rate limits and logging on lookups.
- **Base64 and split fields are heuristics.** Base64 is decoded only when the result is readable text with a digit in it; split fields are joined only for groups of exactly four digits within 24 characters. Other encodings (hex, gzip, ROT13, numbers spelled in words) are not handled; roadmap item 9, the red-team table, should measure them.
- **The judge for E10 uses the same decoders.** “Removed” means a decoder like Aran’s can no longer recover the digits; an attacker with a decoder Aran lacks is not covered.

5. How to run

```
cd research
pip install scikit-learn sklearn-crfsuite cryptography numpy pytest
python -m pytest test_registry.py           # 8 tests, about 2 seconds
./fetch_data.sh && ./fetch_ai_data.sh
python experiment_registry.py               # about 35 minutes the first time (trains the name models),
then about 15
python experiment_registry.py e10           # re-run one experiment
```

Using it from code:

```
        from ai.detector import Detector
        from firewall.aran_firewall import AranFirewall
        from firewall.normalise import Normaliser
        from firewall.registry import CustomerRegistry, OtpRegistry

        reg = CustomerRegistry(key)                 # key from the HSM
        reg.add("AADHAAR", "4914 1777 6311")        # every identifier the bank holds
        otp = OtpRegistry(key, ttl=300)
        otp.issue("482913", trace_id)               # called by the OTP service

        det = Detector(ner, otp_clf, upi_clf, registry=reg, otp_registry=otp, normaliser=Normaliser())
        fw = AranFirewall(det, key)
        clean, findings = fw.process(line, src="crm", sink="log-platform", trace_id=trace_id)
        reg.remove("AADHAAR", "4914 1777 6311")     # DPDP erasure
```

6. What this changes in the roadmap

Items 2 and 5 are done. Item 3 (fixing the ML weak spots directly) is still useful for banks that cannot run a registry, since mobiles fall back to 69.8% and OTPs to 79.4% without one. The next Tier 1 steps are item 4 (crypto-shredding with erasure receipts) and item 1 (the live gateway and dashboard).